

Chapter 14.6

Hyperparameter Calculation

தமிழில்

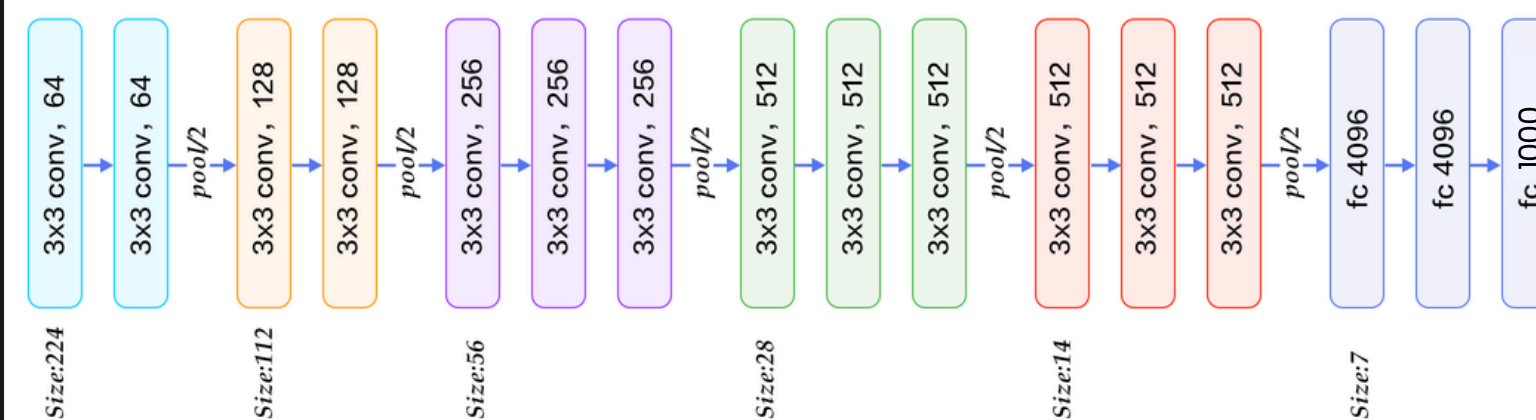
Less than 20 Mins

AI COACH JOHN

CEO & FOUNDER

Strive For Better Future

Calculation	Layer	o/p shape	No. of Parameters
① For 1 conv. layer of 3×3 , $[3 \times 3 \times 3] + 1$ For 16 filters, $28 \times 28 = 1792$	Input layer	$28, 28, 3$	0
	1) Conv 2D	$28, 28, 16$	1792
	2) Conv 2D	$28, 28, 16$	36928
	Maxpool 2D	$14, 14, 16$	0
② For 1 conv. layer of 3×3 , $[3 \times 3 \times 16] + 1$ For 64 filters, $5 \times 5 \times 16 = 36928$	3) Conv 2D	$14, 14, 128$	73856
	4) Conv 2D	$14, 14, 128$	147584
	Maxpool 2D	$7, 7, 128$	0
③ For 1 conv. of 3×3 , $[3 \times 3 \times 64] + 1$ For 128 filters, $5 \times 5 \times 64 = 73856$	5) Conv 2D	$7, 7, 256$	285168
	6) Conv 2D	$7, 7, 256$	590080
	7) Conv 2D	$7, 7, 256$	590080
	Maxpool 2D	$28, 28, 256$	0
④ For 1 conv, $[3 \times 3 \times 128] + 1 = 1153$ For 128 filters, $1153 \times 128 = 147584$	8) Conv 2D	$28, 28, 512$	1180160
	9) Conv 2D	$28, 28, 512$	2359808
	10) Conv 2D	$28, 28, 512$	2359808
⑤ For 1 conv. $[3 \times 3 \times 128] + 1 = 1153$ For 256 filters, $1153 \times 256 = 295148$	Maxpool 2D	$14, 14, 512$	0
	11) Conv 2D	$14, 14, 512$	2359808
	12) Conv 2D	$14, 14, 512$	2359808
	13) Conv 2D	$14, 14, 512$	2359808
⑥ For 1 conv. $[3 \times 3 \times 256] + 1 = 2305$ For 256 filters, $2305 \times 256 = 590080$	Maxpool 2D	$7, 7, 512$	0
	Flatten	25088	0
⑦ Same as ⑥			
⑧ For 1 conv. $[3 \times 3 \times 256] + 1 = 2305$ For 512 filters, $2305 \times 512 = 1180160$	14) Dense	4096 $[4096 \times 25088] + 4096$	102764544
	15) Dense	4096 $[4096 \times 4096] + 4096$	16781312
	16) Dense	1000 $[1000 \times 4096] + 1000$	4097000
⑨ For 1 conv, $[3 \times 3 \times 512] + 1 = 4609$ For 512 filters, $4609 \times 512 = 2359808$			
Tot. no of params = 138,357 in VGG 16			



*But still had **limitations**...*

Heavy Model

- 138 million parameters 🤖
- Huge memory usage
- Slow training

👉 Not efficient for real-world deployment

Tot. no of params in VGG 16 = 138,357,544

Overall Summary from the Beginning

Topics Covered So Far...

1. Why Deep Learning? Inspiration behind it.
2. ANN & its building blocks
3. How **BackPropagation** Works?
4. Activation Functions
5. Optimizers & Loss Functions
6. **Numpy** for Deep Learning
7. MNIST Digit Classification (*Black & White Images*)
 - a. *Modular-Level Coding*
 - b. *How to deploy Locally*
8. ANN for **Regression Problem**
9. ANN for **Classification** (*External Structured Dataset*)
10. Hyper-parameter Tuning
11. CNN Theory & Scripting
12. No Coding AI Tool to build Intelligence... (***Teachable Machines***)

Topics Covered So Far...

13. What is Transfer Learning? Why?

14. **Computer Vision: Evolution of CNN Architectures**

1. Lenet 5 (1998)

2. AlexNet (2012)

3. VGG16 & VGG 19 (2014)

4. Transfer Learning Script for CNN Architectures using Tensorflow and Keras

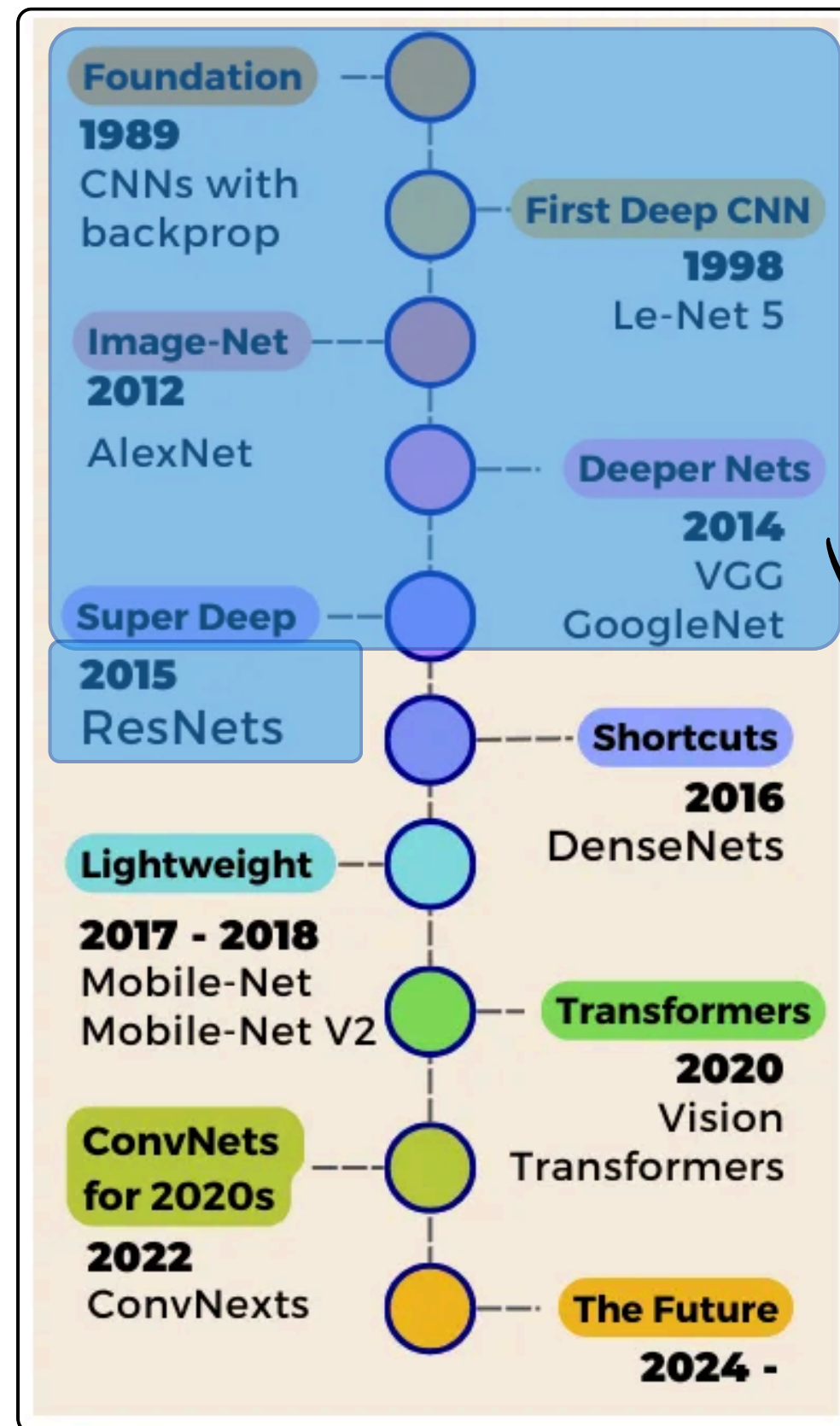
5. HyperParameter Calculation Understanding

What Next?

The **WINNER** (1st Prize) →
"GoogleNet" (*InceptionNet*)
2014 released



Computer Vision Architectures: A Journey of Innovation



Era 1 → **CLASSIC CNN**
(1985 - 2015)

Foundations of Deep Learning

NEXT →

*We will now see
how GoogLeNet Model Works?*